You do the science. The stack does the rest.

Who does what at each step of the campaign, and where you and your agent work together.

optional: report hosted on GitHub Pages

✎ text natural language · ⟨/⟩ code code, by you or your agent · { } data structured data · ⚙ auto generated for you · 1 you approve here

feed forward: the report checks results against what success looks like

	0 Raw data	1 Index & averaged	2a Plan	2b Select	3 Analysis	4 Figures	5 Report
Researchers you: bring the questions and own the science	run the measurements	—	describe the measurements to analyse, what to extract and what success looks like ✎ text	describe the runs you need, pick from the candidates ✎ text 1	choose the fit model and what to extract, check the results ⟨/⟩ code 2	make your figures, in your style ⟨/⟩ code	give the verdict: confirmed, refuted, inconclusive or measurement failed ✎ text 3
⇅ *where you and your agent meet*			⇅ plan together	⇅ find runs	⇅ build the analysis	⇅ make plots	⇅ review results
Agents your assistant: does the legwork on request	never touches raw data	—	turns your words into a PLAN.md section, asks what's missing ✎ text	searches every database, shows you matching runs { } data	writes the analysis code and runs it ⟨/⟩ code	drafts plotting code on request ⟨/⟩ code	builds the report, flags results that disagree with the plan ⚙ auto
Maintainers keep the stack running: nothing to do for you	keep raw data safe and read-only	build the run index and averaged copies ⟨/⟩ code ⚙ auto	PLAN.md format ⟨/⟩ code	index, schema, tests ⟨/⟩ code	base classes, runner, checks ⟨/⟩ code	figure rendering ⟨/⟩ code	Slidev deck, publishing ⟨/⟩ code

You stay in control at three points: 1 **the runs** 2 **the method** 3 **the verdict**

Everything else is handled for you: finding runs, file paths, checks, versions, averaging, rendering figures and building the report.

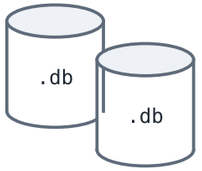
Organising an analysis campaign

Example: is T1 limited by field-dependent loss? From the question to the verdict, each step hands a fixed format to the next.

git code, plan, selections, figures · **drive** raw data, averaged copies, outputs · researchers provide content · *roles: previous slide · formats: next slide →*

0 Raw data

what was measured?



QCoDeS databases, read-only

drive

1 Index

what runs exist?

all runs, plus averaged copies

index: git

copies: drive

2a Plan

what's the question?

```
PLAN.md
## T1 vs field
Status: open
```

question, needs and success, in words

PLAN.md

2b Select

which runs answer it?

```
Q2_wk14.json
✓ run 520   0.5 T
✓ run 524   1 T
✓ run 528   2 T
```

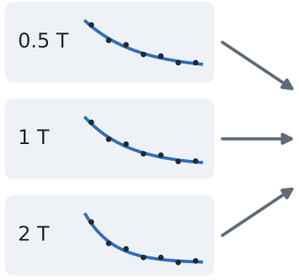
an agent proposes, you confirm

selections/

3 Analysis

runs → numbers, never plots

3a each run on its own



3b all runs together

T1 vs B

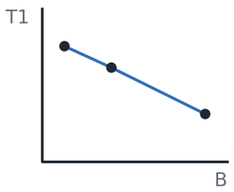
0.5 T	41 µs
1 T	35 µs
2 T	22 µs

+ fit across fields

code: git **outputs: drive**

4 Figures

numbers → plots

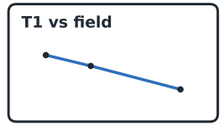


read outputs only, restyle freely

figures/

5 Report

did it work?



confirmed

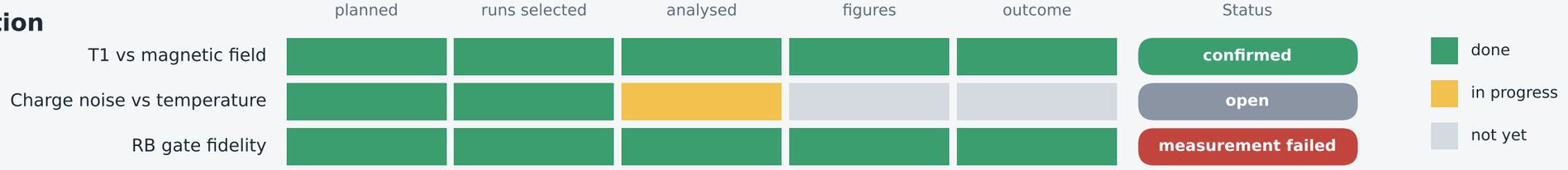
progress & outcome per question

generated

feed forward: the report checks results against what success looks like

Report: progress per question

computed from PLAN.md, selections and outputs; Status set by the owner



Rules

1. Raw data is read-only
2. A run is (db, run_id) + guid, never a bare run_id
3. Each step reads only the step before it
4. Analysis changed → bump VERSION
5. Success criteria before results, verdicts by people

File formats between the steps

The structured files and classes (PLAN.md is natural language). One selection file = one call of the analysis.

1 Selection file

`selections/<analysis>/<name>.json`

```
{
  "analysis": "t1_vs_field",
  "kwargs": {"fit_model": "exp_decay"},
  "runs": [
    {"label": "B_0p5T",
     "db": "setup_a/cooldown_2.db",
     "run_id": 520,
     "guid": "aaaaaaaa-...-01924a01c3e1",
     "modified_dimension": "field_T",
     "value": 0.5,
     "kwargs": {}},
    {"label": "B_1T", ...}
  ]
}
```

analysis	module in analysis/ to call
kwargs	settings, routed by KWARGS
runs	the runs accumulated in this call
each run:	
label	name of the run, unique in file
db, run_id, guid	which run, checked vs. the index
modified_dimension	what is varied, e.g. field_T
value	its value in that unit, or null
kwargs	overrides for this run only

2 Analysis module

`analysis/<analysis>.py`

```
class T1Fit(RunAnalysis):              # 3a
    VERSION = "1.0"
    DATA = "averaged"                  # or "raw"
    KWARGS = {"fit_model": "exp_decay"}

    def analyse(self, run):
        # once per run, on a fresh instance
        # run.data    -> xr.Dataset (measured)
        # self.kwargs -> resolved settings
        ...                            # fit one decay
        return {"data": {"decay": ds},
                "results": {"t1_s": 4.1e-5,
                            "fit_ok": True}}

class T1VsField(AccumulatedAnalysis):  # 3b
    VERSION = "1.0"
    RUN_ANALYSIS = T1Fit

    def accumulate(self, runs):
        # runs[label].results / .data / .value
        ...                            # T1 vs field
        return {"data": {"t1_vs_field": ds},
                "results": {"t1_max_s": 4.1e-5}}
```

kwargs from the selection file

top-level kwargs → each class that declares them in KWARGS

per-run kwargs → the RunAnalysis · undeclared keys are an error

3 Saved outputs

`OUTPUT_ROOT/<analysis>/v<VERSION>/`

analyse and accumulate both return

```
{"data":    {key: xr.Dataset},
 "results": {key: scalar}}
```

saved on the drive by run_analysis()

```
<name>/
├── meta.json
├── runs/
│   ├── B_0p5T/
│   │   ├── results.json
│   │   └── data/decay.nc
│   └── B_1T/ ...
└── accumulated/
    ├── results.json
    └── data/t1_vs_field.nc
```

read in figures with wynaut.io

```
out = load("t1_vs_field", name, version="1.0")
out["runs"]["B_0p5T"]     out["accumulated"]
load_results_table(..., step="run")
```

Checked by run_analysis() on every call · any failure → nothing is saved

before: schema · labels unique · modified_dimension and value both set or both null · every (db, run_id, guid) is in the run index

classes: one AccumulatedAnalysis per module · VERSION in each class's own body · every kwarg declared in KWARGS · labels match REQUIRED_RUNS

after each analyse: only "data" (xr.Dataset values) and "results" (scalars), same results keys for every run after accumulate: the same